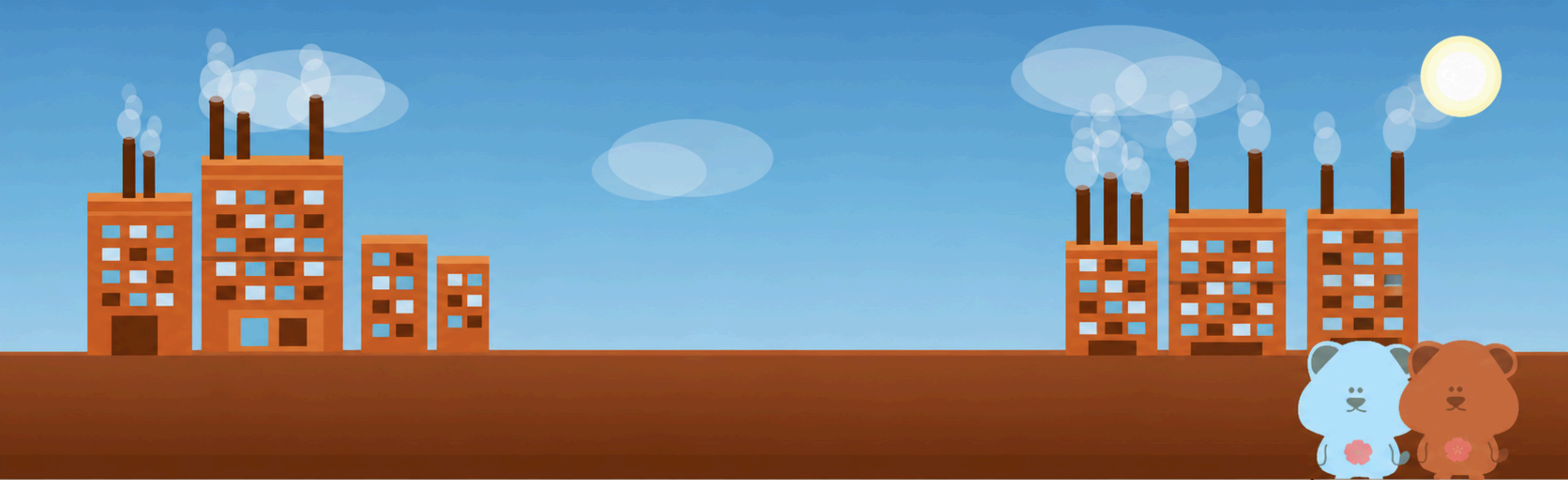




제조업 AI Agent

이재린(2371046), 유다현(2270049), 김서영(2329004)

목차

#01 예지보전(PdM) 도메인 소개

#02 데이터 특성(EDA)

#03 파이프라인 설계

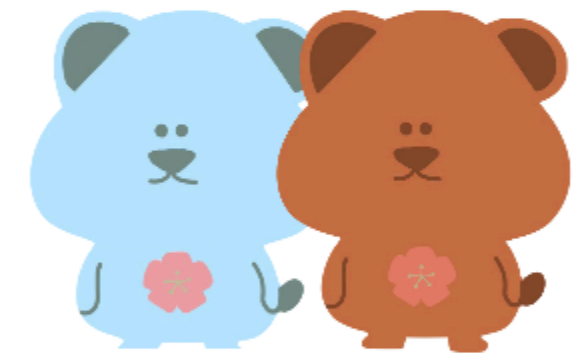
#04 에이전트별 설명

- Agent 1
- Agent 2
- Agent 3
- Agent 4

#05 최종 결과물



#01 예지보전(PHM) 도메인 소개



#01 예지보전(PdM) 도메인 소개

Predictive Maintenance 이란?

- 센서와 AI 기술을 활용해 기계 설비의 상태를 실시간으로 살피고 고장 시기를 미리 예측하여 수리하는 예방 유지보수 방식
- 예지보전 시장: 2022년에 55억 달러로 성장, 2028년까지 연평균 성장률은 17%로 추정.

PdM에 쓰이는 예측 접근법

Physical model-based

물리 법칙 기반 수학적 모델링 → 정밀한 상태-고장 추정 필요

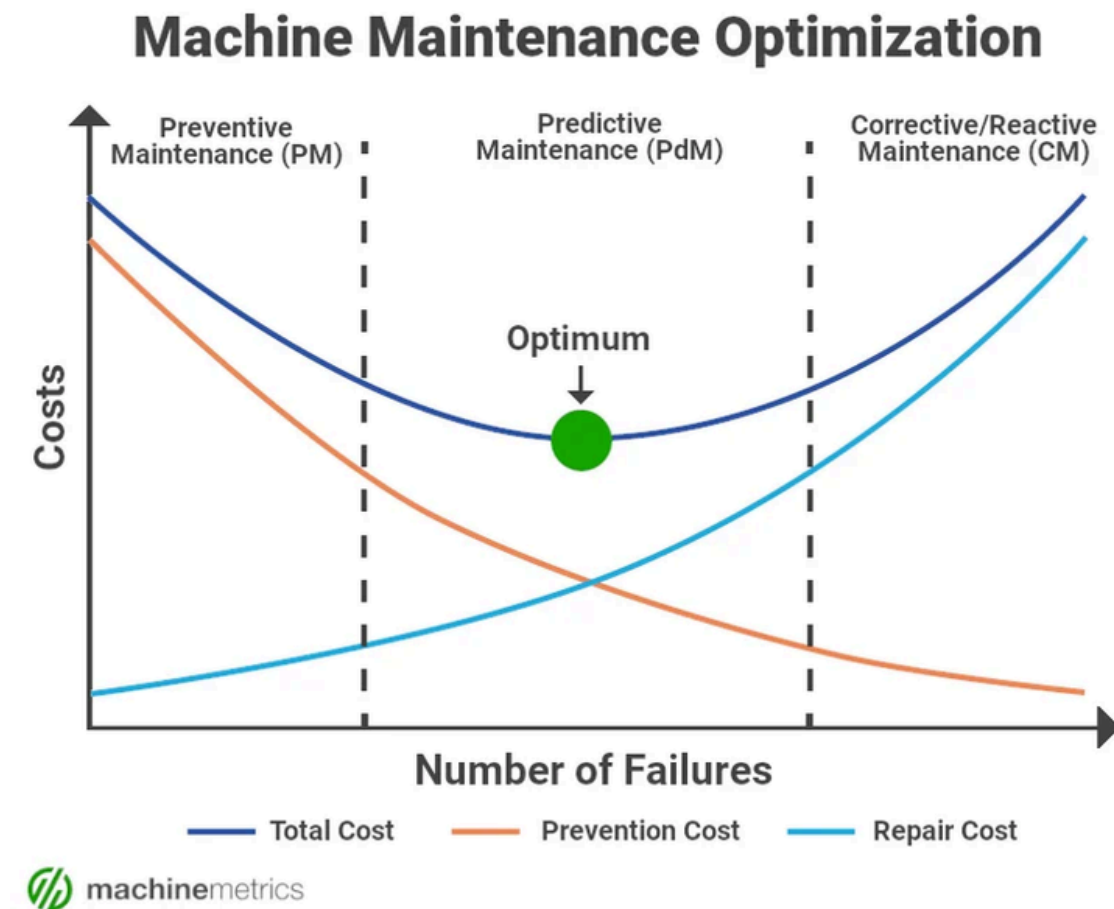
Knowledge-based

전문가 시스템+규칙 기반 → 물리 모델의 복잡도를 낮춘 접근

Data-driven

통계적 패턴인식/ML/AI 기반 → 현재 PdM 솔루션 대부분이 여기 속함

왜 예지보전(PdM)이 필요한가



15~60%

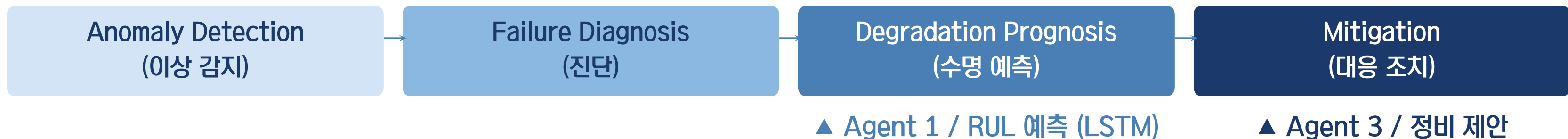
정비(maintenance) 비용이
전체 제조 운영 비용에서 차지하는 비중

Zonta et al. (2020), Computers & Industrial Engineering

#01 예지보전(PdM) 도메인 소개

PdM 4단계 로드맵 & Agent 구조 매핑

Serradilla et al. (2021). Applied Intelligence,



prognostics 방식 두 가지

분류(Classification) 기반

- 고장 여부를 이진(0/1)으로 판별
- '언제' 고장이 날지 시점 정보 부재
- 임계값 설정에 따라 결과 변동 큼
- 조기 경보 타이밍 조절이 어려움

★ 회귀(Regression) 기반

RUL 값을 실수값으로 직접 예측해,
더욱 자연스럽게 구체적인 추세 판단 가능

- 잔여 수명(RUL)을 연속값으로 출력
- 시간 축 위에서 열화 경향 시각화
- 정비 시점을 정량적으로 제안 가능

#01 예지보전(PdM) 도메인 소개

LSTM 채택 & 불확실성 정량화

★ LSTM 채택 근거

- Serradilla et al. (2021)
PdM 딥러닝 리뷰에서 LSTM이 시계열 센서 데이터의 장기 의존성 포착에 가장 적합한 구조로 평가됨
- Jamshidi et al. (2025)
다변량 시계열 예측 벤치마크에서 LSTM 기반 모델이 고장 징후 조기 포착 정확도 상위권 달성
- Pan et al. (2025)
RUL 예측 태스크에서 LSTM이 열화 경향의 비선형 패턴을 안정적으로 학습함을 실증

RUL 예측의 불확실성 정량화 — Agent 1의 CI 설계

Pan et al. (2025) Artificial Intelligence for Engineering Design, Analysis and Manufacturing (AI EDAM)

Aleatoric
데이터/측정 잡음

Epistemic
모델/파라미터 불확실성

★ MC Dropout + 베이지안 신경망(deep Gaussian process)

- Agent 1은 Dropout을 유지한 채 N회 반복 샘플링하여 예측 분포의 평균과 분산을 추정.
- 점 추정이 아닌 신뢰구간(CI)을 출력하여, 정비 의사결정에 불확실성 정보를 함께 제공.

#02 데이터 특성(EDA)



#02 데이터 특성(EDA)

데이터셋 개요 : PHM2010 CNC Milling Tool Wear

실험 대상

3-flute HSS 초경 공구의 6개 (c1~6) → 학습용 C1, C4, C6

샘플링 주파수

50,000 Hz 고주파 품질 데이터 (현미경 개별 파일)

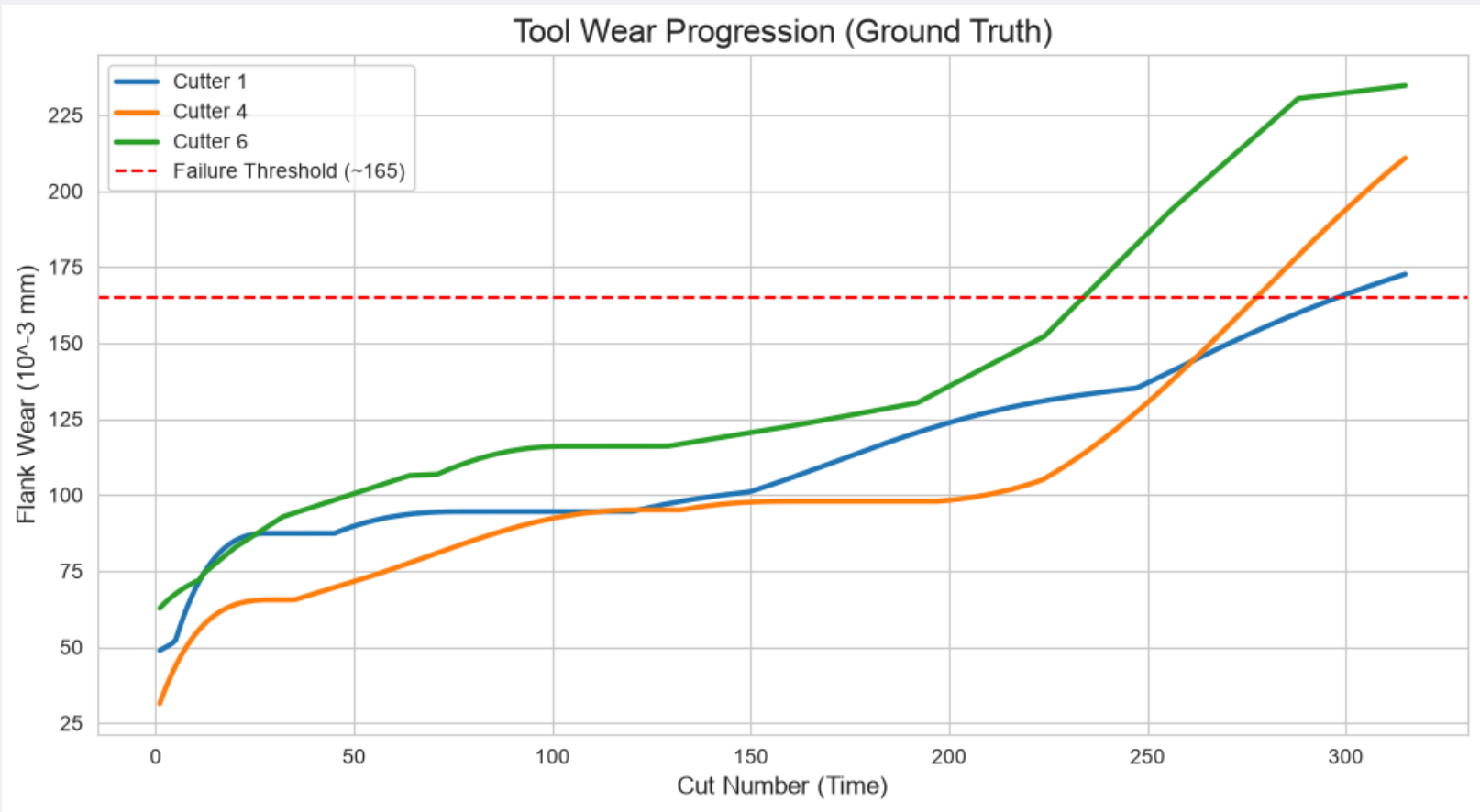
센서 세트 (7종)

Force x,y,z, Vibration x,y,z, AE_rms

라벨(ground truth)

Flank Wear(마모량, mm) — Threshold 165 μ m 도달 시 EOL

공구 마모 진행 곡선 (c1, c4, c6)



커터마다 수명 도달 속도가 다름 → manufacturing variance 존재

#02 데이터 특성(EDA)

RUL 이란 ?

왜 마모값이 아니라 RUL 을 예측하는가

RUL (Remaining Useful Life, 잔여 유효 수명)

$$t_{fail} = \min\{t : \bar{w}(t) \geq 150\} \quad RUL(t) = \max(t_{fail} - t, 0) = \text{cut 단위 정수}$$

$W(t) = \text{mean}(\text{flute1}, \text{flute2}, \text{flute3})$ 마모값 · EOL threshold = 150 (최종 공정 종료, 최대 허용)

마모값 (예: 현재 120 μ m)

- 지금 얼마가 남았는지는 알 수 없음
- 이전 공정과 동일하게 보는 X는 필요 없음
- 단순 수치만으로는 교체 시점 판단 불가



★ RUL

- 남은 작업 횟수(cut 수)를 직접 예측
- RUL 값은 정수 / cut 단위로 해석
- Agent 2(판단 Agent)가 바로 임계값 기반 의사결정
(● 즉시 교체 / ● 집중 모니터링 / ● 정상 운전)에 사용할 수 있음.

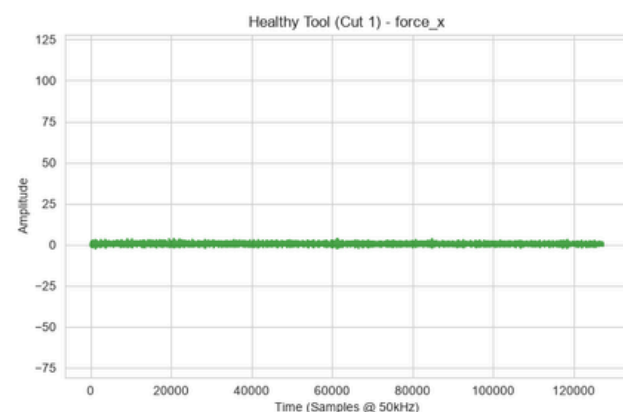
예시 $t_{fail} = 228$, 현재 $t = 200 \rightarrow RUL = 228 - 200 = 28$ (cut)

#02 데이터 특성(EDA)

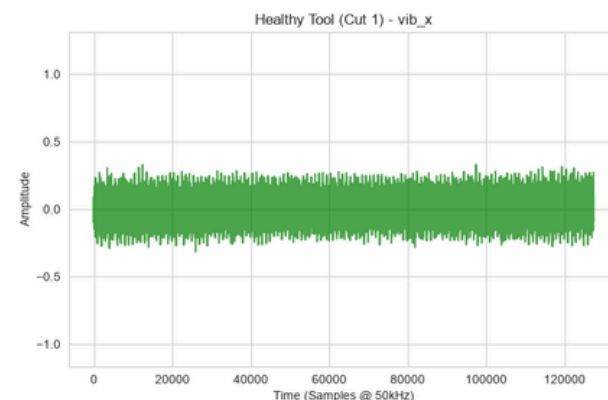
센서 신호는 실제로 마모를 반영하는가

Cut #1 — Healthy Tool

force_x 진폭이 작고 규칙적

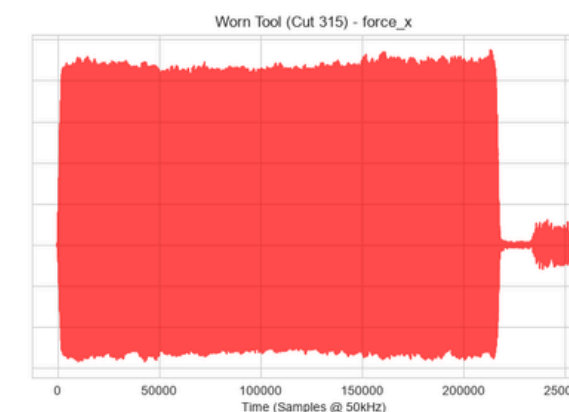


vib_x 안정적인 진동

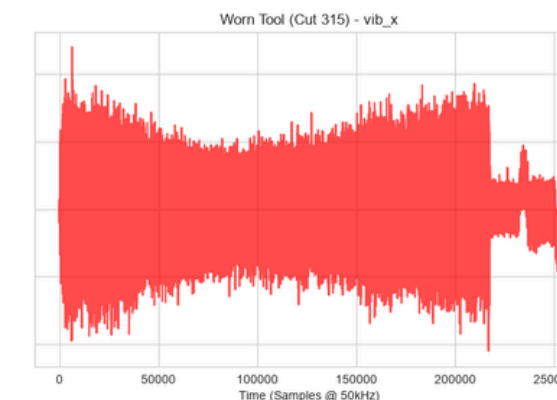


Cut #315 — Worn Tool

force_x 진폭이 크고 불규칙적 (절삭 저항 증가)



vib_x 진동 증가 + 노이즈 발생



주파수 영역(FFT)에서도 동일한 패턴 확인

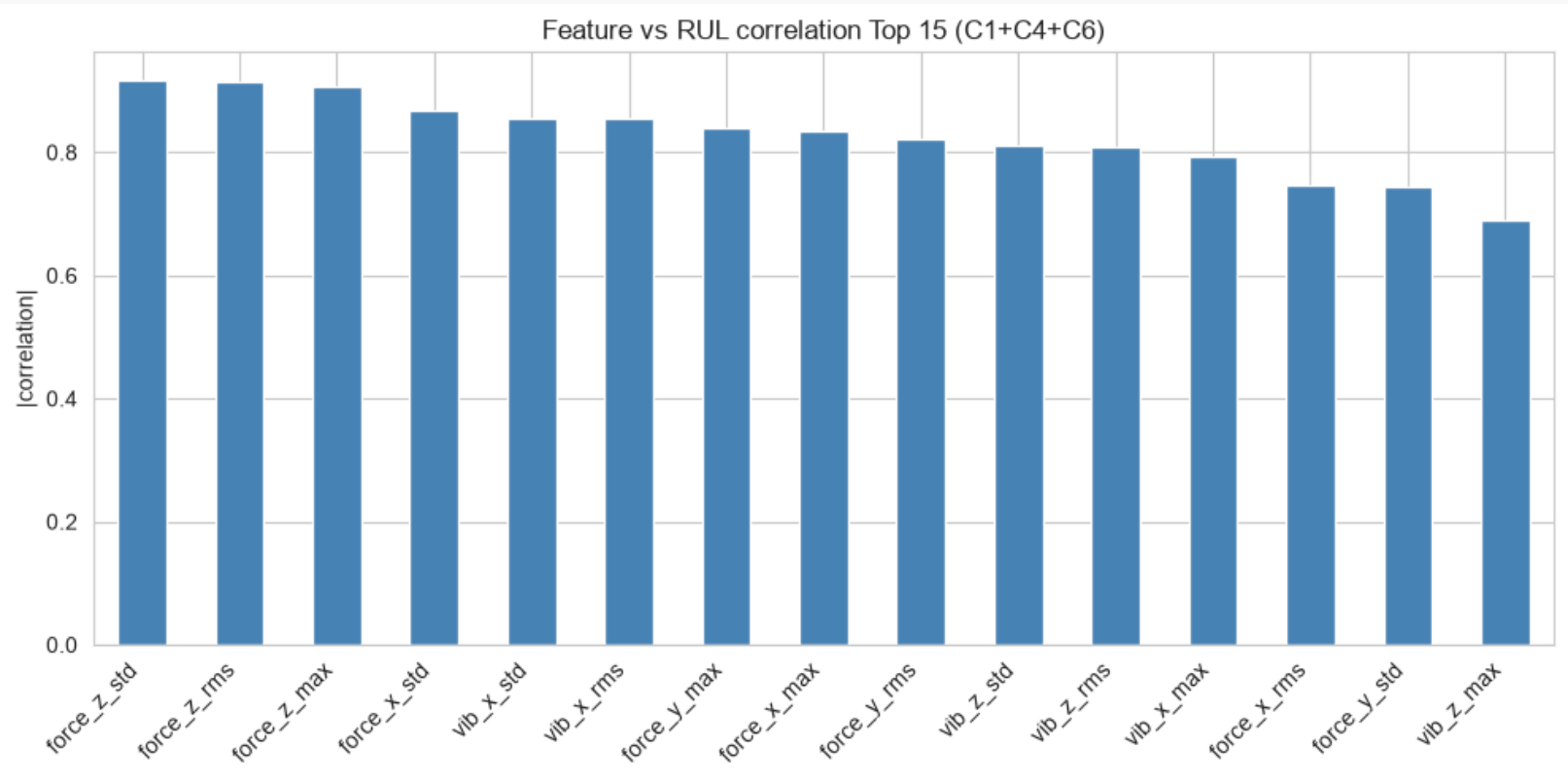
New Tool vs Worn Tool 스펙트럼 비교 결과, 마모가 진행될수록 특정 주파수 대역(~520Hz 부근) 진폭 변화 관찰

⇒ 결론: 센서 신호가 진동(mag)으로 마모가 실제로 반영 → 센서 데이터로 RUL 예측이 가능하다는 전제 성립

#02 데이터 특성(EDA)

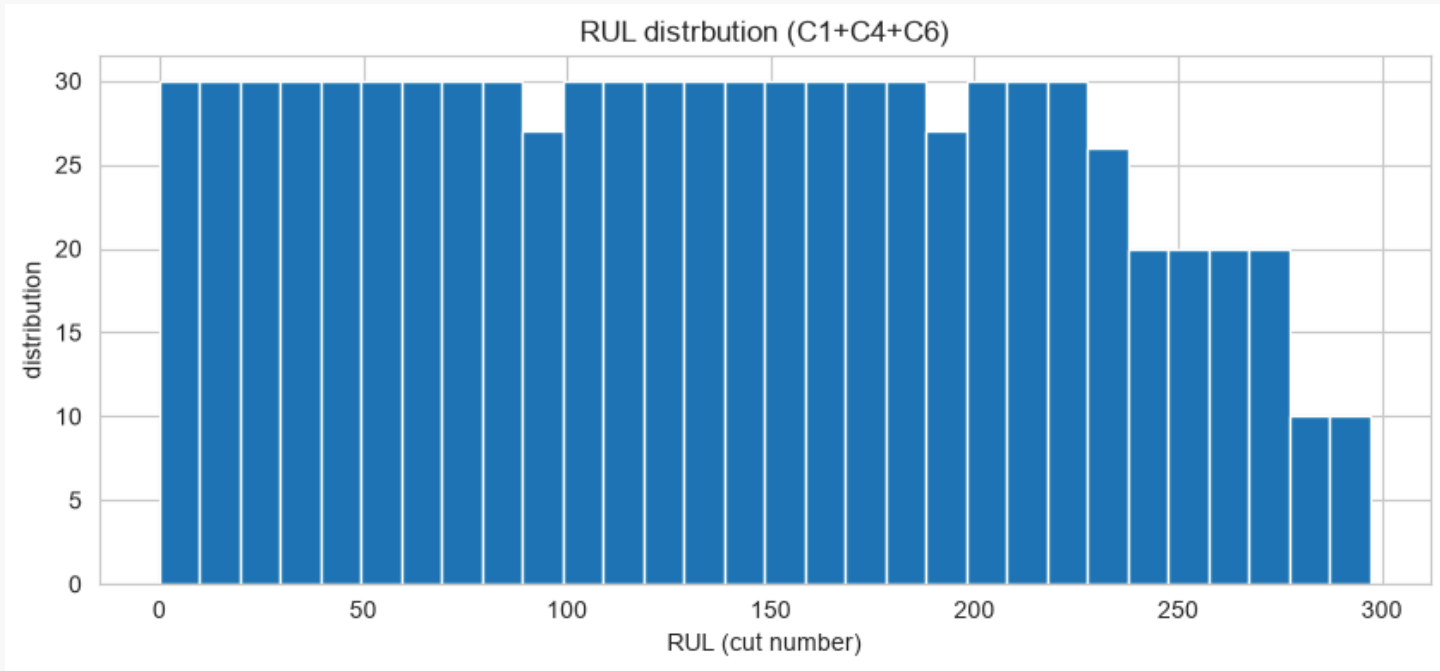
Feature 선택 근거 & RUL 라벨 분포

Feature vs RUL 상관관계 (lcorr 상위, 개략)



Force_Z 계열이 압도적 1~3위 → Z축(축방향) 절삭력이 마모와 가장 강하게 연동 · AE_RMS는 Top15 밖 (제거 고려)

RUL 라벨 분포 (C1 + C4 + C6)



⚠ 클래스 불균형 (Class Imbalance)

y_test의 30.8%가 이미 RUL=0 → PdM 분야의 공통 obstacle (Jamshidi 2025, Pan 2025) — Agent 1 설계·향후 계획에 반영

#03 파이프라인 설계



#03 파이프라인 설계

[Agent 전체 파이프라인 및 라우팅 조건]



- Agent① 예측 → Agent② 판정 → Agent③ 정비계획(조건부) → Agent④ 리포트, 4단계 순차 파이프라인
- 핵심 설계 포인트: Agent③은 항상 실행되지 않고 조건부로만 실행됨
 - 판정이 "위험"/"주의"면 항상 호출
 - 판정이 "정상"이어도 임계값 경계선 근처(± 10)면 호출 — 매뉴얼 5부(경계선 근처 보정 규칙) 반영
 - 그 외("정상" & 경계선 아님)는 Agent③ 스킵, 바로 Agent④로

#03 파이프라인 설계

[LangGraph의 StateGraph로 구현]

agent3_planner.py

```
def should_invoke_agent3(state: dict[str, Any]) -> bool:
    """
    Agent 3를 호출해야 하는지 판단.
    - action이 "위험" 또는 "주의"면 항상 True.
    - action이 "정상"이어도 is_near_boundary=True면 True (5부 보정 규칙).
    """
    action = state.get("action")
    near = bool(state.get("is_near_boundary", False))

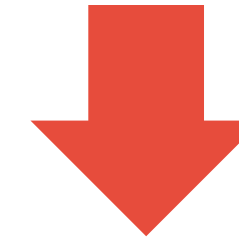
    if action in ("위험", "주의"):
        return True
    if action == "정상" and near:
        return True
    return False
```

agent3에서 호출 여부 판단 및 graph에 전달

graph.py

```
# 조건부 라우팅

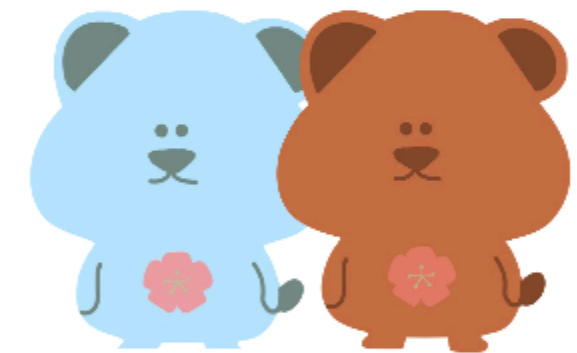
def route_after_agent2(state: PipelineState) -> str:
    """Agent 2 판정 직후 분기점.
    agent3_planner.should_invoke_agent3()를 그대로 재사용 - 판단 로직을
    이 파일에서 다시 구현하지 않고 단일 소스(agent3_planner.py)만 따른다.
    """
    return "agent3" if should_invoke_agent3(state) else "agent4_skip"
```



```
graph.add_conditional_edges(
    "agent2",
    route_after_agent2,
    {
        "agent3": "agent3",          # 위험/주의 또는 정상+경계선근처
        "agent4_skip": "agent4",    # 정상 && 경계선 아님 -> Agent 3 건너뛰
    },
)
```

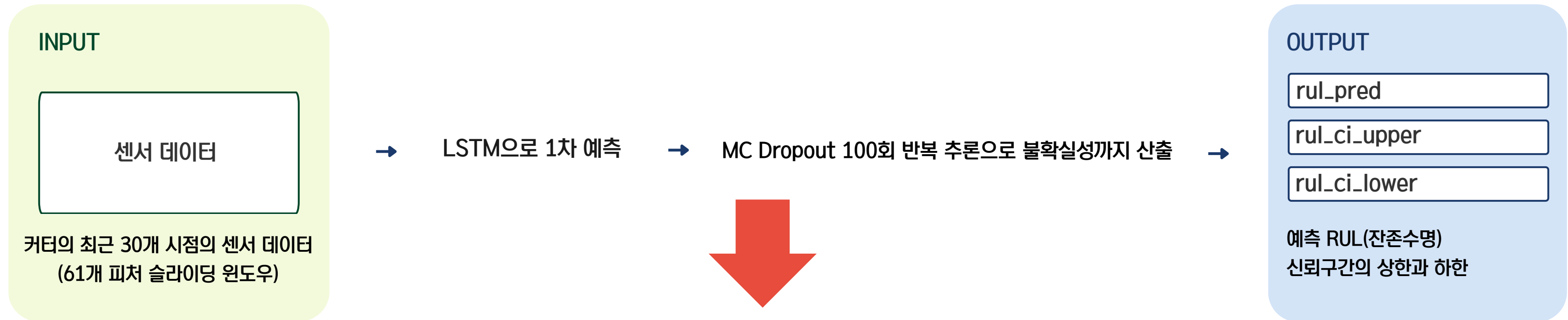
agent3 값을 바탕으로 graph에서 조건부 라우팅

#04 에이전트별 설명



#04 Agent 1 — 예측 Agent

agent1_predictor.py : LSTM + MC Dropout 기반 잔존수명 예측

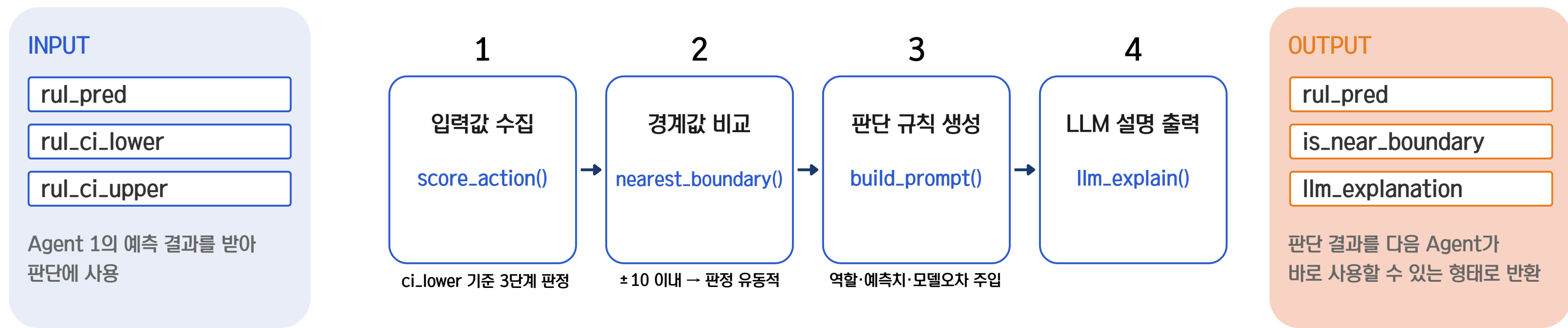


```
class LSTMPredictor(nn.Module):
    def __init__(self, input_size=61, hidden_size=64, num_layers=2, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                             dropout=dropout, batch_first=True)

        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_size, 1)

    def predict_with_uncertainty(model, x, n_iter=100):
        model.train() # dropout 활성화한 채로 추론 (MC Dropout 핵심)
        preds = [model(x_t) for _ in range(n_iter)]
        rul_pred = preds.mean()
        ci_lower, ci_upper = percentile(preds, [5, 95])
```

#04 Agent 2 — 판단 Agent



임계값 (실측 percentile 기반)

RED_THRESHOLD = 20.7
ci_lower ≤ 20.7 → 위험 (하위 10%)

YELLOW_THRESHOLD = 50.25
ci_lower ≤ 50.25 → 주의 (하위 25%)

BOUNDARY_MARGIN = ± 10
경계값 근처 → 판정 유동적 표시

핵심 설계 포인트 — LLM이 예측값을 과신하지 않도록 강제

① 모델 오차 강제 주입

Test RMSE 14.41 / MAE 9.02를 프롬프트에 '반드시 반영' 지시

② 도메인 갭 명시

C6은 학습(C1+C4)에 없던 cutter
→ "신뢰도 낮음"을 설명에 명시하도록 강제

③ 경계선 근처 플래그

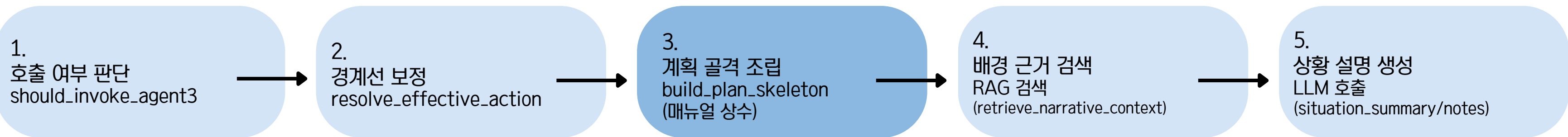
MC Dropout의 확률적 흔들림으로 판정이 뒤집힐 수 있음을 is_near_boundary로 별도 표시

④ 원본 수치 보존

→ 숫자 3종 그대로 전달

#04 Agent 3 - 정비 계획 Agent (Planner)

매뉴얼에 이미 확정된 정비 계획을 조립하고, LLM은 이번 케이스의 '상황 설명'만 자연어로 채우는 역할의 Agent



1단계 상세 - 언제 호출 되는가?

위험 / 주의
항상 호출

정상 + 경계선 근처 = 최종 주의 등급
호출 (경계선 보정 규칙 적용)

정상 + 경계선 아님
호출 안 함 → Agent 4 “정비 계획 없음” 처리

설계 내용

1. 경계선 보정 규칙
"정상"이어도 경계선 근처면 자동으로 "주의" 상향 적용 → 주의는 위험으로 상향 적용
2. 역할 분리 (확정값 vs 생성값)
부품·절차·소요시간은 코드 상수 그대로, LLM은 상황 요약만 생성
3. RAG 역할 한정
"왜 이 임계값인지" 배경 설명만 검색. 부품명·시간 수치는 절대 RAG에서 가져오지 않음
4. 프롬프트에 "재생성 금지" 명시
계획 골격은 참고만, 설명에만 자연스럽게 녹이도록 제약 - LLM의 임의 수치 생성 방지

출력 & 보존 근거

maintenance_plan + maintenance_plan_raw_context(RAG 원문 → Agent 4 전달) · original_action/effective_action
= 둘 다 남겨 상향 조치 근거 유지

#04 Agent 4 - 리포트 Agent (Reporter)

Agent 1,2,3 결과를 모아 LLM으로 최종 리포트를 작성하고 TXT/PDF로 저장하는 역할의 Agent

입력
Agent 1 + 2 + 3 결과
(통합된 State)

1.
프롬프트 조립
build_reporter_user_prompt

2.
리포트 생성
LLM 호출 (call_llm_test)

3.
파일 저장
save_report_txt/pdf

매뉴얼 근거 리포트에 반영하기

Agent 3에서 가져온 것
maintenance_plan_raw_context (RAG 검색 원문)

이 근거를 리포트에 반영하기 위해 한 것

- 1) 가공 함수 신규 작성 - format_manual_reference_section()
검색 결과 없으면 안내 문구, 너무 길면 잘라서 반환
- 2) 프롬프트에 실제로 연결
build_reporter_user_prompt()에 호출을 추가해 raw_context를
LLM에게 실제로 전달
- 3) 인용 규칙 명시
판정 결과 및 근거 섹션 안에서만, 최소 1건 인용, 원본 파일명 노출 금지

Formal / Quick 모드

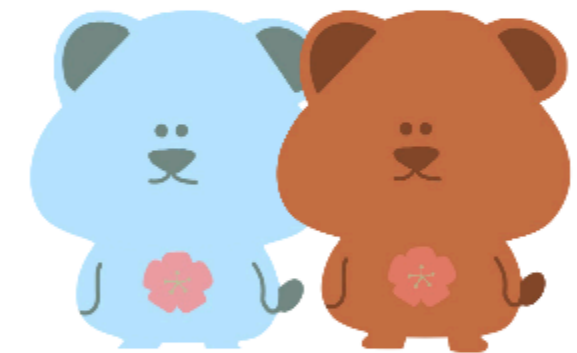
Formal 모드

- 상사 보고용 정식 문서
- 문체: 완결된 "-습니다"체
 - 매뉴얼 배경 설명 인용 0 (근거 섹션 1건)
 - 파일명 접미사 없음

Quick 모드

- 현장 확인용 요약
- 문체: 화살표(→)·가운뎃점(·) 개조식
 - 매뉴얼 배경 설명 인용 X (생략)
 - 파일명에 _quick 접미사 자동 부여

#05 최종 결과물



#05 최종 결과물

전체 파이프라인 연결 검증 결과_4가지 케이스 실제 실행 검증

case	판정	Agent 3	비고
정상 & 경계선 아님	정상	스킵	바로 Agent 4로, "정비 계획 없음" 자동 처리
정상 + 경계선근처	정상→주의(보정)	호출	5부 규칙에 따라 한 단계 상향, escalated=True
주의	주의	호출	RAG로 매뉴얼 배경 설명 인용
위험	위험	호출	정비 절차 5단계 자동 조립 + PDF/TXT 리포트 생성

EWCHA
EUROPEAN

```
(.venv) (base) bluecloud@ijaelin-ui-MacBookAir predictive-maintenance-agent % python main.py 220
device: mps
[디버그] 실행 파일 : /Users/bluecloud/workspace_vscode/vscode_research/predictive-maintenance-agent/main.py
[디버그] 사용할 index: 220
[Agent1] RUL 예측 시작 (MC Dropout 100회)...
[Agent1] 완료 -> rul_pred=0.29
[Agent2] 판정 + LLM 설명 생성 시작 (Ollama 첫 호출은 모델 로딩 때문에 느릴 수 있음)...
[Agent2] 완료 -> action=위험
[Agent3] 정비 계획 생성 시작 (RAG 검색 + LLM 호출)...
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 199/199 [00:00<00:00, 5384.95it/s]
[Agent3] 완료
[Agent4] 리포트 생성 시작 (LLM 호출, formal 모드는 max_tokens=10000이라 특히 오래 걸릴 수 있음)...
[Agent4] 완료
cutter_id: c6
실제 RUL: 0.0
예측 RUL: 0.29 [0.00, 0.79]
판정: 위험 (경계선 근처: False)
설명: CNC 밀링 공구의 예측 사용시간(RUL)은 0.29 사이클로 나타나며, 이는 검증용 데이터에서 평균 31.1 사이클의 오차를 보였음을 고려할 때 주의가 필요합니다. 95% 신뢰 구간이 [0.00, 0.79]으로 매우 넓은 범위를 나타내므로, 이는 모델의 알려진 예측 불확실성과 일치합니다.
정비 계획: effective_action=위험 escalated=False
상황 요약: 커터 c6의 예측 RUL은 0.2898778021335602 사이클로, 95% 신뢰구간은 [0.0, 0.785316526889801]을 나타내며 매우 넓게 퍼져 있음. 이는 모델의 예측 불확실성을 반영하며, Agent 2 판정에 따라 '위험' 등급으로 분류됨. 경계선 근처 판정은 거리가 떨어져 있으므로 별도 조치 없이 즉시 교체 절차를 시작하여 당일 내 완료 목표로 설정함.
리포트 저장 경로: {'txt': 'reports/phm_report_c6_20260730_161711.txt', 'pdf': 'reports/phm_report_c6_20260730_161711.pdf'}
```

#05 최종 결과물

[실제 위험 케이스 리포트 발췌_정비 절차]

4. 정비 절차

- 안전조치/전원 차단 (2~5분)
- 공구 탈거 (1~2분)
- 테이퍼 청소 및 신품 삽입 (5~10분)
- 오프셋 설정 (2.5~20분)
- 검증(Proving-out) (5~30분)

예상 총 소요 시간: 약 15.5~67분

- 부품/절차/토크값 등 확정 수치는 매뉴얼 원문 (manual_data.py)에서 그대로 조립 — LLM이 숫자를 새로 생성하지 않도록 역할 분리
- RAG(ChromaDB + 로컬 한국어 임베딩)로 "왜 이 임계값인지" 같은 배경 근거만 검색 — 확정 수치와 서술을 분리해 환각 위험 차단

THANK YOU

